the contract stated that the payout would be made for a block when the block passed the customer's acceptance test. The details of those acceptance tests were not specified in the contract.

During the course of this project, we worked very closely with the customer. We released the software to him almost every Friday. By Monday or Tuesday of the following week, he would have a list of changes for us to put into the software. We would prioritize those changes together and then schedule them into subsequent weeks. The customer worked so closely with us that acceptance tests were never an issue. He knew when a block of functionality satisfied his needs because he watched it evolve from week to week.

The requirements for this project were in a constant state of flux. Major changes were not uncommon. There were whole blocks of functionality that were removed and others that were inserted. Yet the contract, and the project, survived and succeeded. The key to this success was the intense collaboration with the customer and a contract that governed that collaboration rather than trying to specify the details of scope and schedule for a fixed cost.

Responding to change over following a plan. It is the ability to respond to change that often determines the success or failure of a software project. When we build plans, we need to make sure that our plans are flexible and ready to adapt to changes in the business and technology.

The course of a software project cannot be planned very far into the future. First of all, the business environment is likely to change, causing the requirements to shift. Second, customers are likely to alter the requirements once they see the system start to function. Finally, even if we know the requirements, and we are sure they won't change, we are not very good at estimating how long it will take to develop them.

It is tempting for novice managers to create a nice PERT or Gantt chart of the whole project and tape it to the wall. They may feel that this chart gives them control over the project. They can track the individual tasks and cross them off the chart as they are completed. They can compare the actual dates with the planned dates on the chart and react to any discrepancies.

What *really* happens is that the structure of the chart degrades. As the team gains knowledge about the system, and as the customers gain knowledge about their needs, certain tasks on the chart become unnecessary. Other tasks will be discovered and will need to be added. In short, the plan will undergo changes in *shape*, not just changes in dates.

A better planning strategy is to make detailed plans for the next two weeks, very rough plans for the next three months, and extremely crude plans beyond that. We should know the tasks we will be working on for the next two weeks. We should roughly know the requirements we will be working on for the next three months. And we should have only a vague idea what the system will do after a year.

This decreasing resolution of the plan means that we are only investing in a detailed plan for those tasks that are immediate. Once the detailed plan is made, it is hard to change since the team will have a lot of momentum and commitment. However, since that plan only governs a few weeks' worth of time, the rest of the plan remains flexible.

Principles

The above values inspired the following 12 principles, which are the characteristics that differentiate a set of agile practices from a heavyweight process:

- *Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.*

  The *MIT Sloan Management Review* published an analysis of software development practices that help companies build high-quality products.[3] The article found a number of practices that had a significant impact on the quality of the final system. One practice was a strong correlation between quality and the early deliv-

3. *Product-Development Practices That Work: How Internet Companies Build Software*, MIT Sloan Management Review, Winter 2001, Reprint number 4226.

ery of a partially functioning system. The article reported that *the less functional the initial delivery, the higher the quality in the final delivery.*

Another finding of this article is a strong correlation between final quality and frequent deliveries of increasing functionality. *The more frequent the deliveries, the higher the final quality.*

An agile set of practices delivers early and often. We strive to deliver a rudimentary system within the first few weeks of the start of the project. Then, we strive to continue to deliver systems of increasing functionality every two weeks.

Customers may choose to put these systems into production if they think that they are functional enough. Or they may choose simply to review the existing functionality and report on changes they want made.

- *Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.*

  This is a statement of attitude. The participants in an agile process are not afraid of change. They view changes to the requirements as *good* things, because those changes mean that the team has learned more about what it will take to satisfy the market.

  An agile team works very hard to keep the structure of its software flexible so that when requirements change, the impact to the system is minimal. Later in this book we will learn the principles and patterns of object-oriented design that help us to maintain this kind of flexibility.

- *Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter time scale.*

  We deliver *working* software, and we delivery it early (after the first few weeks) and often (every few weeks thereafter). We are not content with delivering bundles of documents or plans. We don't count those as true deliveries. Our eye is on the goal of delivering software that satisfies the customer's needs.

- *Business people and developers must work together daily throughout the project.*

  In order for a project to be agile, there must be significant and frequent interaction between the customers, developers, and stakeholders. A software project is not like a fire-and-forget weapon. A software project must be continuously guided.

- *Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.*

  An agile project is one in which people are considered the most important factor of success. All other factors—process, environment, management, etc.—are considered to be second order effects, and they are subject to change if they are having an adverse effect upon the people.

  For example, if the office environment is an obstacle to the team, the office environment must be changed. If certain process steps are an obstacle to the team, the process steps must be changed.

- *The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.*

  In an agile project, people *talk* to each other. The primary mode of communication is conversation. Documents may be created, but there is no attempt to capture all project information in writing. An agile project team does not demand written specs, written plans, or written designs. Team members may create them if they perceive an immediate and significant need, but they are not the default. The default is conversation.

- *Working software is the primary measure of progress.*

  Agile projects measure their progress by measuring the amount of software that is currently meeting the customer's need. They don't measure their progress in terms of the phase that they are in or by the volume of documentation that has been produced or by the amount of infrastructure code they have created. They are 30% done when 30% of the necessary functionality is working.

- *Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.*

  An agile project is not run like a 50-yard dash; it is run like a marathon. The team does not take off at full speed and try to maintain that speed for the duration. Rather, they run at a fast, but sustainable, pace.

  Running too fast leads to burnout, shortcuts, and debacle. Agile teams pace themselves. They don't allow themselves to get too tired. They don't borrow tomorrow's energy to get a bit more done today. They work at a rate that allows them to maintain the highest quality standards for the duration of the project.
- *Continuous attention to technical excellence and good design enhances agility.*

  High quality is the key to high speed. The way to go fast is to keep the software as clean and robust as possible. Thus, all agile team members are committed to producing only the highest quality code they can. They do not make messes and then tell themselves they'll clean it up when they have more time. If they make a mess, they clean it up before they finish for the day.
- *Simplicity—the art of maximizing the amount of work not done—is essential.*

  Agile teams do not try to build the grand system in the sky. Rather, they always take the simplest path that is consistent with their goals. They don't put a lot of importance on anticipating tomorrow's problems, nor do they try to defend against all of them today. Instead, they do the simplest and highest-quality work today, confident that it will be easy to change if and when tomorrow's problems arise.
- *The best architectures, requirements, and designs emerge from self-organizing teams.*

  An agile team is a self-organizing team. Responsibilities are not handed to individual team members from the outside. Responsibilities are communicated to the team as a whole, and the team determines the best way to fulfill them.

  Agile team members work together on all aspects of the project. Each is allowed input into the whole. No single team member is responsible for the architecture or the requirements or the tests. The team shares those responsibilities, and each team member has influence over them.
- *At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.*

  An agile team continually adjusts its organization, rules, conventions, relationships, etc. An agile team knows that its environment is continuously changing and knows that they must change with that environment to remain agile.

Conclusion

The professional goal of every software developer and every development team is to deliver the highest possible value to their employers and customers. And yet our projects fail, or fail to deliver value, at a dismaying rate. Though well intentioned, the upward spiral of process inflation is culpable for at least some of this failure. The principles and values of agile software development were formed as a way to help teams break the cycle of process inflation and to focus on simple techniques for reaching their goals.

At the time of this writing, there were many agile processes to choose from. These include SCRUM,[4] Crystal,[5] Feature Driven Development,[6] Adaptive Software Development (ADP),[7] and most significantly, Extreme Programming.[8]

4. `www.controlchaos.com`
5. `crystalmethodologies.org`
6. *Java Modeling In Color With UML: Enterprise Components and Process*, Peter Coad, Eric Lefebvre, and Jeff De Luca, Prentice Hall, 1999.
7. [Highsmith2000].
8. [Beck1999], [Newkirk2001].

Bibliography

1. Beck, Kent. *Extreme Programming Explained: Embracing Change*. Reading, MA: Addison–Wesley, 1999.
2. Newkirk, James, and Robert C. Martin. *Extreme Programming in Practice*. Upper Saddle River, NJ: Addison–Wesley, 2001.
3. Highsmith, James A. *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems*. New York, NY: Dorset House, 2000.

2

Overview of Extreme Programming



As developers we need to remember that XP is not the only game in town.

—Pete McBreen

The previous chapter gave us an outline of what agile software development is about. However, it didn't tell us exactly what to do. It gave us some platitudes and goals, but it gave us little in the way of real direction. This chapter corrects that.

The Practices of Extreme Programming

Extreme programming is the most famous of the agile methods. It is made up of a set of simple, yet interdependent practices. These practices work together to form a whole that is greater than its parts. We shall briefly consider that whole in this chapter, and examine some of the parts in chapters to come.

Customer Team Member

We want the customer and developers to work closely with each other so that they are both aware of each other's problems and are working together to solve those problems.

Who is the customer? The customer of an XP team is the person or group who defines and prioritizes features. Sometimes, the customer is a group of business analysts or marketing specialists working in the same company as the developers. Sometimes, the customer is a user representative commissioned by the body of users. Sometimes the customer is in fact the paying customer. But in an XP project, whoever the customers are, they are members of, and available to, the team.

The best case is for the customer to work in the same room as the developers. Next best is if the customer works in within 100 feet of the developers. The larger the distance, the harder it is for the customer to be a true team member. If the customer is in another building or another state, it is very difficult to integrate him or her into the team.

What do you do if the customer simply cannot be close by? My advice is to find someone who *can* be close by and who is willing and able to stand in for the true customer.

User Stories

In order to plan a project, we must know something about the requirements, but we don't need to know very much. For planning purposes, we only need to know enough about a requirement to estimate it. You may think that in order to estimate a requirement you need to know all its details, but that's not quite true. You have to know that there *are* details, and you have to know roughly the kinds of details there are, but you don't have to know the specifics.

The specific details of a requirement are likely to change with time, especially once the customer begins to see the system come together. There is nothing that focuses requirements better than seeing the nascent system come to life. Therefore, capturing the specific details about a requirement long before it is implemented is likely to result in wasted effort and premature focusing.

When using XP, we get the sense of the details of the requirements by talking them over with the customer, but we do not capture that detail. Rather, the customer writes *a few words* on an index card that we agree will remind us of the conversation. The developers write an estimate on the card at roughly the same time that the customer writes it. They base that estimate on the sense of detail they got during their conversations with the customer.

A user story is a mnemonic token of an ongoing conversation about a requirement. It is a planning tool that the customer uses to schedule the implementation of a requirement based upon its priority and estimated cost.

Short Cycles

An XP project delivers working software every two weeks. Each of these two-week iterations produces working software that addresses some of the needs of the stakeholders. At the end of each iteration, the system is demonstrated to the stakeholders in order to get their feedback.

The Iteration Plan. An iteration is usually two weeks in length. It represents a minor delivery that may or may not be put into production. It is a collection of user stories selected by the customer according to a budget established by the developers.

The developers set the budget for an iteration by measuring how much they got done in the previous iteration. The customer may select any number of stories for the iteration, so long as the total of their estimates does not exceed that budget.

Once an iteration has been started, the customer agrees not to change the definition or priority of the stories in that iteration. During this time, the developers are free to cut the stories up in to *tasks* and to develop the tasks in the order that makes the most technical and business sense.

The Release Plan. XP teams often create a release plan that maps out the next six iterations or so. That plan is known as a release plan. A release is usually three months worth of work. It represents a major delivery that can usually be put into production. A release plan consists of prioritized collections of user stories that have been selected by the customer according to a budget given by the developers.

The developers set the budget for the release by measuring how much they got done in the previous release. The customer may select any number of stories for the release so long as the total of the estimates does not exceed that budget. The customer also determines the order in which the stories will be implemented in the release. If the team so desires, they can map out the first few iterations of the release by showing which stories will be completed in which iterations.

Releases are not cast in stone. The customer can change the content at any time. He or she can cancel stories, write new stories, or change the priority of a story.

Acceptance Tests

The details about the user stories are captured in the form of acceptance tests specified by the customer. The acceptance tests for a story are written immediately preceding, or even concurrent with, the implementation of that story. They are written in some kind of scripting language that allows them to be run automatically and repeatedly. Together, they act to verify that the system is behaving as the customers have specified.

The language of the acceptance tests grows and evolves with the system. The customers may recruit the developers to create a simple scripting system, or they may have a separate quality assurance (QA) department that can develop it. Many customers enlist the help of QA in developing the acceptance-testing tool and with writing the acceptance tests themselves.

Once an acceptance test passes, it is added to the body of passing acceptance tests and is never allowed to fail again. This growing body of acceptance tests is run several times per day, every time the system is built. If an acceptance tests fails, the build is declared a failure. Thus, once a requirement is implemented, it is never broken. The system migrates from one working state to another and is never allowed to be inoperative for longer than a few hours.

Pair Programming

All *production* code is written by pairs of programmers working together at the same workstation. One member of each pair drives the keyboard and types the code. The other member of the pair watches the code being typed, looking for errors and improvements.[1] The two interact intensely. Both are completely engaged in the act of writing software.

The roles change frequently. The driver may get tired or stuck, and his pair partner will grab the keyboard and start to drive. The keyboard will move back and forth between them several times in an hour. The resultant code is designed and authored by both members. Neither can take more than half the credit.

Pair membership changes at least once per day so that every programmer works in two different pairs each day. Over the course of an iteration, every member of the team should have worked with every other member of the team, and they should have worked on just about everything that was going on in the iteration.

This dramatically increases the spread of knowledge through the team. While specialties remain and tasks that require certain specialties will usually belong to the appropriate specialists, those specialists will pair with nearly everyone else on the team. This will spread the specialty out through the team such that other team members can fill in for the specialists in a pinch.

Studies by Laurie Williams[2] and Nosek[3] have suggested that pairing does not reduce the efficiency of the programming staff, yet it significantly reduces the defect rate.

1. I have seen pairs in which one member controls the keyboard and the other controls the mouse.
2. [Williams2000], [Cockburn2001].
3. [Nosek].

Test-Driven Development

Chapter 4, which is on testing, discusses test-driven development in great detail. The following paragraphs provide a quick overview.

All production code is written in order to make failing unit tests pass. First we write a unit test that fails because the functionality for which it is testing doesn't exist. Then we write the code that makes that test pass.

This iteration between writing test cases and code is very rapid, on the order of a minute or so. The test cases and code evolve together, with the test cases leading the code by a very small fraction. (See "A Programming Episode" in Chapter 6 for an example.)

As a result, a very complete body of test cases grows along with the code. These tests allow the programmers to check whether the program works. If a pair makes a small change, they can run the tests to ensure that they haven't broken anything. This greatly facilitates *refactoring* (discussed later).

When you write code in order to make test cases pass, that code is, by definition, testable. In addition, there is a strong motivation to decouple modules from each other so that they can be independently tested. Thus, the design of code that is written in this fashion tends to be much less coupled. The principles of object-oriented design play a powerful role in helping you with this decoupling.[4]

Collective Ownership

A pair has the right to check out *any* module and improve it. No programmers are individually responsible for any one particular module or technology. Everybody works on the GUI.[5] Everybody works on the middleware. Everybody works on the database. Nobody has more authority over a module or a technology than anybody else.

This doesn't mean that XP denies specialties. If your specialty is the GUI, you are most likely to work on GUI tasks, but you will also be asked to pair on middleware and database tasks. If you decide to learn a second specialty, you can sign up for tasks and work with specialists who will teach it to you. You are not confined to your specialty.

Continuous Integration

The programmers check in their code and integrate several times per day. The rule is simple. The first one to check in wins, everybody else merges.

XP teams use nonblocking source control. This means that programmers are allowed to check any module out at any time, regardless of who else may have it checked out. When the programmer checks the module back in after modifying it, he must be prepared to merge it with any changes made by anyone who checked the module in ahead of him. To avoid long merge sessions, the members of the team check in their modules very frequently.

A *pair* will work for an hour or two on a task. They create test cases and production code. At some convenient breaking point, probably long before the task is complete, the pair decides to check the code back in. They first make sure that all the tests run. They integrate their new code into the existing code base. If there is a merge to do, they do it. If necessary, they consult with the programmers who beat them to the check in. Once their changes are integrated, they build the new system. They run every test in the system, including all currently running acceptance tests. If they broke anything that used to work, they fix it. Once all the tests run, they finish the check in.

Thus, XP teams will build the system many times each day. They build the *whole* system from end to end.[6] If the final result of a system is a CD, they cut the CD. If the final result of the system is an active Web site, they install that Web site, probably on a testing server.

4. See Section II.
5. I'm not advocating a three-tiered architecture here. I just chose three common partitions of software technology.
6. Ron Jeffries says, "End to end is farther than you think."

Sustainable Pace

A software project is not a sprint; it is a marathon. A team that leaps off the starting line and starts racing as fast as it can will burn out long before they are close to finishing. In order to finish quickly, the team must run at a sustainable pace; it must conserve its energy and alertness. It must intentionally run at a steady, moderate pace.

The XP rule is that a team is not *allowed* to work overtime. The only exception to that rule is the last week in a release. If the team is within striking distance of its release goal and can sprint to the finish, then overtime is permissible.

Open Workspace

The team works together in an open room. There are tables set up with workstations on them. Each table has two or three such workstations. There are two chairs in front of each workstation for pairs to sit in. The walls are covered with status charts, task breakdowns, UML diagrams, etc.

The sound in this room is a low buzz of conversation. Each pair is within earshot of every other. Each has the opportunity to hear when another is in trouble. Each knows the state of the other. The programmers are in a position to communicate intensely.

One might think that this would be a distracting environment. It would be easy to fear that you'd never get anything done because of the constant noise and distraction. In fact, this doesn't turn out to be the case. Moreover, instead of interfering with productivity, a University of Michigan study suggested that working in a "war room" environment may *increase* productivity by a factor of two.[7]

The Planning Game



The next chapter, "Planning," goes into great detail about the XP planning game. I'll describe it briefly here.

The essence of the planning game is the division of responsibility between business and development. The business people (a.k.a. the customers) decide how important a feature is, and the developers decide how much that feature will cost to implement.

At the beginning of each release and each iteration, the developers give the customers a budget, based on how much they were able to get done in the last iteration or in the last release. The customers choose stories whose costs total up to, but do not exceed that budget.

With these simple rules in place, and with short iterations and frequent releases, it won't be long before the customers and developers get used to the rhythm of the project. The customers will get a sense for how fast the developers are going. Based on that sense, the customers will be able to determine how long their project will take and how much it will cost.

Simple Design

An XP team makes their designs as simple and expressive as they can be. Furthermore, they narrow their focus to consider only the stories that are planned for the current iteration. They don't worry about stories to come. Instead, they migrate the design of the system, from iteration to iteration, to be the best design for the stories that the system currently implements.

This means that an XP team will probably not start with infrastructure. They probably won't select the database first. They probably won't select the middleware first. The team's first act will be to get the first batch of stories working in the *simplest way possible*. The team will only add the infrastructure when a story comes along that forces them to do so.

7. http://www.sciencedaily.com/releases/2000/12/001206144705.htm

The following three XP mantras guide the developer:

Consider the Simplest Thing That Could Possibly Work. XP teams always try to find the simplest possible design option for the current batch of stories. If we can make the current stories work with flat files, we might not use a database or EJB. If we can make the current stories work with a simple socket connection, we might not use an ORB or RMI. If we can make the current stories work without multithreading, we might not include mutithreading. We try to consider the simplest way to implement the current stories. Then we choose a solution that is as close to that simplicity as we can *practically* get.

You Aren't Going to Need It. Yeah, but we *know* we're going to need that database one day. We *know* we're going to need an ORB one day. We *know* we're going to have to support multiple users one day. So we need to put the hooks in for those things *now*, don't we?

An XP team seriously considers what will happen if they resist the temptation to add infrastructure before it is strictly needed. They start from the assumption that they aren't going to need that infrastructure. The team puts in the infrastructure, only if they have proof, or at least very compelling evidence, that putting in the infrastructure now will be more cost effective than waiting.

Once and Only Once. XPers don't tolerate code duplication. Wherever they find it, they eliminate it.

There are many sources of code duplication. The most obvious are those stretches of code that were captured with a mouse and plopped down in multiple places. When we find those, we eliminate them by creating a function or a base class. Sometimes two or more algorithms may be remarkably similar, and yet they differ in subtle ways. We turn those into functions or employ the TEMPLATE METHOD pattern.[8] Whatever the source of duplication, once discovered, we won't tolerate it.

The best way to eliminate redundancy is to create abstractions. After all, if two things are similar, there must be some abstraction that unifies them. Thus, the act of eliminating redundancy forces the team to create many abstractions and further reduce coupling.

Refactoring[9]

I cover this topic in more detail in Chapter 5. What follows is a brief overview.

Code tends to rot. As we add feature after feature and deal with bug after bug, the structure of the code degrades. Left unchecked, this degradation leads to a tangled, unmaintainable mess.

XP teams reverse this degradation through frequent refactoring. Refactoring is the practice of making a series of tiny transformations that improve the structure of the system without affecting its behavior. Each transformation is trivial, hardly worth doing. But together, they combine into significant transformations of the design and architecture of the system.

After each tiny transformation, we run the unit tests to make sure we haven't broken anything. Then we do the next transformation and the next and the next, running the tests after each. In this manner we keep the system working while transforming its design.

Refactoring is done continuously rather than at the end of the project, the end of the release, the end of the iteration, or even the end of the day. Refactoring is something we do every hour or every half hour. Through refactoring, we continuously keep the code as clean, simple, and expressive as possible.

Metaphor

Metaphor is the least understood of all the practices of XP. XPers are pragmatists at heart, and this lack of concrete definition makes us uncomfortable. Indeed, the proponents of XP have often discussed removing metaphor as a practice. And yet, in some sense, metaphor is one of the most important practices of all.

8. See Chapter 14, "Template Method & Strategy: Inheritance v. Delegation."
9. [Fowler99].

Think of a jigsaw puzzle. How do you know how the pieces go together? Clearly, each piece abuts others, and its shape must be perfectly complimentary to the pieces it touches. If you were blind and you had a very good sense of touch, you could put the puzzle together by diligently sifting through each piece and trying it in position after position.

But there is something more powerful than the shape of the pieces binding the puzzle together. There is a picture. The picture is the true guide. The picture is so powerful that if two adjacent pieces of the picture do not have complementary shapes, then you *know* that the puzzle maker made a mistake.

That is the metaphor. It's the big picture that ties the whole system together. It's the vision of the system that makes the location and shape of all the individual modules obvious. If a module's shape is inconsistent with the metaphor, then you know it is the module that is wrong.

Often a metaphor boils down to a system of names. The names provide a vocabulary for elements in the system and help to define their relationships.

For example, I once worked on a system that transmitted text to a screen at 60 characters per second. At that rate, a screen fill could take some time. So we'd allow the program that was generating the text to fill a buffer. When the buffer was full, we'd swap the program out to disk. When the buffer got close to empty, we'd swap the program back in and let it run more.

We spoke about this system in terms of dump trucks hauling garbage. The buffers were little trucks. The display screen was the dump. The program was the garbage producer. The names all fit together and helped us think about the system as a whole.

As another example, I once worked on a system that analyzed network traffic. Every thirty minutes, it would poll dozens of network adapters and pull down the monitoring data from them. Each network adapter gave us a small block of data composed of several individual variables. We called these blocks "slices." The slices were raw data that needed to be analyzed. The analysis program "cooked" the slices, so it was called "The Toaster." We called the individual variables within the slices, "crumbs." All in all, it was a useful and entertaining metaphor.

Conclusion

Extreme programming is a set of simple and concrete practices that combines into an agile development process. That process has been used on many teams with good results.

XP is a good general-purpose method for developing software. Many project teams will be able to adopt it as is. Many others will be able to adapt it by adding or modifying practices.

Bibliography

1. Dahl, Dijkstra. *Structured Programming*. New York: Hoare, Academic Press, 1972.
2. Conner, Daryl R. *Leading at the Edge of Chaos*. Wiley, 1998.
3. Cockburn, Alistair. *The Methodology Space*. Humans and Technology technical report HaT TR.97.03 (dated 97.10.03), `http://members.aol.com/acockburn/papers/methyspace/methyspace.htm`.
4. Beck, Kent. *Extreme Programming Explained: Embracing Change*. Reading, MA: Addison–Wesley, 1999.
5. Newkirk, James, and Robert C. Martin. *Extreme Programming in Practice*. Upper Saddle River, NJ: Addison–Wesley, 2001.
6. Williams, Laurie, Robert R. Kessler, Ward Cunningham, Ron Jeffries. *Strengthening the Case for Pair Programming*. IEEE Software, July–Aug. 2000.
7. Cockburn, Alistair, and Laurie Williams. *The Costs and Benefits of Pair Programming*. XP2000 Conference in Sardinia, reproduced in *Extreme Programming Examined*, Giancarlo Succi, Michele Marchesi. Addison–Wesley, 2001.
8. Nosek, J. T. *The Case for Collaborative Programming*. Communications of the ACM (1998): 105–108.
9. Fowler, Martin. *Refactoring: Improving the Design of Existing Code*. Reading, MA: Addison–Wesley, 1999.

3

Planning



"When you can measure what you are speaking about, and express it in numbers, you know something about it; but when you cannot measure it, when you cannot express it in numbers, your knowledge is of a meager and unsatisfactory kind."

—Lord Kelvin, 1883

What follows is a description of the planning game from Extreme Programming (XP).[1] It is similar to the way planning is done in several of the other agile[2] methods like SCRUM,[3] Crystal,[4] feature-driven development,[5] and adaptive software development (ADP).[6] However, none of those processes spell it out in as much detail and rigor.

1. [Beck99], [Newkirk2001].
2. `www.AgileAlliance.org`
3. `www.controlchaos.com`
4. `crystalmethodologies.org`
5. *Java Modeling In Color With UML: Enterprise Components and Process* by Peter Coad, Eric Lefebvre, and Jeff De Luca, Prentice Hall, 1999.
6. [Highsmith2000].

Initial Exploration

At the start of the project, the developers and customers try to identify all the really significant user stories they can. However, they don't try to identify *all* user stories. As the project proceeds, the customers will continue to write new user stories. The flow of user stories will not shut off until the project is over.

The developers work together to estimate the stories. The estimates are relative, not absolute. We write a number of "points" on a story card to represent the relative cost of the story. We may not be sure just how much time a story point represents, but we do know that a story with eight points will take twice as long as a story with four points.

Spiking, Splitting, and Velocity

Stories that are too large or too small are hard to estimate. Developers tend to underestimate large stories and overestimate small ones. Any story that is too big should be split into pieces that aren't too big. Any story that is too small should be merged with other small stories.

For example, consider the story, "Users can securely transfer money into, out of, and between their accounts." This is a big story. Estimating will be hard and probably inaccurate. However, we can split it as follow, into many stories that are much easier to estimate:

- Users can log in.
- Users can log out.
- Users can deposit money into their account.
- Users can withdraw money from their account.
- Users can transfer money from their account to another account.

When a story is split or merged, it should be reestimated. It is not wise to simply add or subtract the estimate. The main reason to split or merge a story is to get it to a size where estimation is accurate. It is not surprising to find that a story estimated at five points breaks up into stories that add up to ten! Ten is the more accurate estimate.

Relative estimates don't tell us the absolute size of the stories, so they don't help us determine when to split or merge them. In order to know the true size of a story, we need a factor that we call *velocity*. If we have an accurate velocity, we can multiply the estimate of any story by the velocity to get the actual time estimate for that story. For example, if our velocity is "2 days per story point," and we have a story with a relative estimate of four points, then the story should take eight days to implement.

As the project proceeds, the measure of velocity will become ever more accurate because we'll be able to measure the number of story points completed per iteration. However, the developers will probably not have a very good idea of their velocity at the start. They must create an initial guess by whatever means they feel will give the best results. The need for accuracy at this point is not particularly grave, so they don't need to spend an inordinate amount of time on it. Often, it is sufficient to spend a few days prototyping a story or two to get an idea of the team's velocity. Such a prototype session is called a *spike*.

Release Planning

Given a velocity, the customers can get an idea of the cost of each of the stories. They also know the business value and priority of each story. This allows them to choose the stories they want done first. This choice is not purely a matter of priority. Something that is important, but also expensive, may be delayed in favor of something that is less important but much less expensive. Choices like this are *business* decisions. The business folks decide which stories give them the most bang for the buck.

The developers and customers agree on a date for the first release of the project. This is usually a matter of 2–4 months in the future. The customers pick the stories they want implemented within that release and the rough

order in which they want them implemented. The customers cannot choose more stories than will fit according to the current velocity. Since the velocity is initially inaccurate, this selection is crude. But accuracy is not very important at this point in time. The release plan can be adjusted as velocity becomes more accurate.

Iteration Planning

Next, the developers and customers choose an iteration size. This is typically two weeks long. Once again, the customers choose the stories that they want implemented in the first iteration. They cannot choose more stories than will fit according to the current velocity.

The order of the stories within the iteration is a technical decision. The developers implement the stories in the order that makes the most technical sense. They may work on the stories serially, finishing each one after the next, or they may divvy up the stories and work on them all concurrently. It's entirely up to them.

The customers cannot change the stories in the iteration once the iteration has begun. They are free to change or reorder any other story in the project, but not the ones that the developers are currently working on.

The iteration ends on the specified date, *even if all the stories aren't done*. The estimates for all the completed stories are totaled, and the velocity for that iteration is calculated. This measure of velocity is then used to plan the next iteration. The rule is very simple. The planned velocity for each iteration is the measured velocity of the previous iteration. If the team got 31 story points done last iteration, then they should plan to get 31 story points done in the next. Their velocity is 31 points per iteration.

This feedback of velocity helps to keep the planning in sync with the team. If the team gains in expertise and skill, the velocity will rise commensurately. If someone is lost from the team, the velocity will fall. If an architecture evolves that facilitates development, the velocity will rise.

Task Planning

At the start of a new iteration, the developers and customers get together to plan. The developers break the stories down into development tasks. A task is something that one developer can implement in 4–16 hours. The stories are analyzed, with the customers' help, and the tasks are enumerated as completely as possible.

A list of the tasks is created on a flip chart, whiteboard, or some other convenient medium. Then, one by one, the developers sign up for the tasks they want to implement. As each developer signs up for a task, he or she estimates that task in arbitrary task points.[7]

Developers may sign up for any kind of task. Database guys are not constrained to sign up for database tasks. GUI guys can sign up for database tasks if they like. This may seem inefficient, but as you'll see, there is a mechanism that manages this. The benefit is obvious. The more the developers know about the *whole* project, the healthier and more informed the project team is. We want knowledge of the project to spread through the team irrespective of specialty.

Each developer knows how many task points he or she managed to implement in the last iteration. This number is their personal *budget*. No one signs up for more points than they have in their budget.

Task selection continues until either all tasks are assigned or all developers have used their budgets. If there are tasks remaining, then the developers negotiate with each other, trading tasks based on their various skills. If this doesn't make enough room to get all the tasks assigned, then the developers ask the customers to remove tasks or stories from the iteration. If all the tasks are signed up and the developers still have room in their budgets for more work, they ask the customers for more stories.

7. Many developers find it helpful to use "perfect programming hours" as their task points.

The Halfway Point



Halfway through the iteration, the team holds a meeting. At this point, half of the *stories* scheduled for the iteration should be complete. If half the stories aren't complete, then the team tries to reapportion tasks and responsibilities to ensure that all the stories will be complete by the end of the iteration. If the developers cannot find such a reapportionment, then the customers need to be told. The customers may decide to pull a task or story from the iteration. At the very least, they will name the lowest priority tasks and stories so that the developers avoid working on them.

For example, suppose the customers selected eight stories totalling 24 story points for the iteration. Suppose also that these were broken down into 42 tasks. At the halfway point of the iteration, we would expect to have 21 tasks and 12 story points complete. Those 12 story points must represent wholly completed stories. Our goal is to complete stories, not just tasks. The nightmare scenario is to get to the end of the iteration with 90% of the tasks complete, but no stories complete. At the halfway point, we want to see completed stories that represent half the story points for the iteration.

Iterating

Every two weeks, the current iteration ends and the next begins. At the end of each iteration, the current running executable is demonstrated to the customers. The customers are asked to evaluate the look, feel, and performance of the project. They will provide their feedback in terms of new user stories.

The customers see progress frequently. They can measure velocity. They can predict how fast the team is going, and they can schedule high-priority stories early. In short, they have all the data and control they need to manage the project to their liking.

Conclusion

From iteration to iteration and release to release, the project falls into a predictable and comfortable rhythm. Everyone knows what to expect and when to expect it. Stakeholders see progress frequently and substantially. Rather than being shown notebooks full of diagrams and plans, they are shown working software that they can touch, feel, and provide feedback on.

Developers see a reasonable plan based upon their own estimates and controlled by their own measured velocity. They choose the tasks on which they feel comfortable working and keep the quality of their workmanship high.

Managers receive data every iteration. They use this data to control and manage the project. They don't have to resort to pressure, threats, or appeals to loyalty to meet an arbitrary and unrealistic date.

If this sounds like blue sky and apple pie, it's not. The stakeholders won't always be happy with the data that the process produces, especially not at first. Using an agile method does not mean that the stakeholders will get what they want. It simply means that they'll be able to control the team to get the most business value for the least cost.

Bibliography

1. Beck, Kent. *Extreme Programming Explained: Embrace Change*. Reading, MA: Addison–Wesley, 1999.
2. Newkirk, James, and Robert C. Martin. *Extreme Programming in Practice*. Upper Saddle River, NJ: Addison–Wesley, 2001.
3. Highsmith, James A. *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems*. New York: Dorset House, 2000.

4

Testing



Fire is the test of gold; adversity, of strong men.

—Seneca (c. 3 B.C.–A.D. 65)

The act of writing a unit test is more an act of design than of verification. It is also more an act of documentation than of verification. The act of writing a unit test closes a remarkable number of feedback loops, the least of which is the one pertaining to verification of function.

Test Driven Development

What if we designed our tests before we designed our programs? What if we refused to implement a function in our programs until there was a test that failed because that function wasn't present? What if we refused to add *even a single line* of code to our programs unless there were a test that was failing because of its absence? What if we incrementally added functionality to our programs by first writing failing tests that asserted the existence of that functionality, and then made the test pass? What effect would this have on the design of the software we were writing? What benefits would we derive from the existence of such a comprehensive bevy of tests?

The first and most obvious effect is that every single function of the program has tests that verify its operation. This suite of tests acts as a backstop for further development. It tells us whenever we inadvertently break some existing functionality. We can add functions to the program, or change the structure of the program, without

fear that we will break something important in the process. The tests tell us that the program is still behaving properly. We are thus much freer to make changes and improvement to our program.

A more important, but less obvious, effect is that the act of writing the test first forces us into a different point of view. We must view the program we are about to write from the vantage point of a caller of that program. Thus, we are immediately concerned with the interface of the program as well as its function. By writing the test first, we design the software to be *conveniently callable.*

What's more, by writing the test first, we force ourselves to design the program to be *testable.* Designing the program to be callable and testable is remarkably important. In order to be callable and testable, the software has to be decoupled from its surroundings. Thus, the act of writing tests first *forces us to decouple the software*!

Another important effect of writing tests first is that the tests act as an invaluable form of documentation. If you want to know how to call a function or create an object, there is a test that shows you. The tests act as a suite of examples that help other programmers figure out how to work with the code. This documentation is compileable and executable. It will stay current. It cannot lie.

An Example of Test-First Design

I recently wrote a version of *Hunt the Wumpus*, just for fun. This program is a simple adventure game in which the player moves through a cave trying to kill the Wumpus before the Wumpus eats him. The cave is a set of rooms that are connected to each other by passageways. Each room may have passages to the north, south, east, or west. The player moves about by telling the computer which direction to go.

One of the first tests I wrote for this program was `testMove` in Listing 4-1. This function creates a new `WumpusGame`, connects room 4 to room 5 via an east passage, places the player in room 4, issues the command to move east, and then asserts that the player should be in room 5.

Listing 4-1

```
public void testMove()
{
  WumpusGame g = new WumpusGame();
  g.connect(4,5,"E");
  g.setPlayerRoom(4);
  g.east();
  assertEquals(5, g.getPlayerRoom());
}
```

All this code was written before any part of `WumpusGame` was written. I took Ward Cunningham's advice and wrote the test the way I wanted it to read. I trusted that I could make the test pass by writing the code that conformed to the structure implied by the test. This is called *intentional programming.* You state your intent in a test before you implement it, making your intent as simple and readable as possible. You trust that this simplicity and clarity points to a good structure for the program.

Programming by intent immediately led me to an interesting design decision. The test makes no use of a `Room` class. The action of *connecting* one room to another communicates my intent. I don't seem to need a `Room` class to facilitate that communication. Instead, I can just use integers to represent the rooms.

This may seem counter intuitive to you. After all, this program may appear to you to be all about rooms; moving between rooms; finding out what rooms contain; etc. Is the design implied by my intent flawed because it lacks a `Room` class?

I could argue that the concept of connections is far more central to the `Wumpus` game than the concept of room. I could argue that this initial test pointed out a good way to solve the problem. Indeed, I think that is the case, but it is not the point I'm trying to make. The point is that the test illuminated a central design issue at a very early stage. *The act of writing tests first is an act of discerning between design decisions.*

Notice that the test tells you how the program works. Most of us could easily write the four named methods of `WumpusGame` from this simple specification. We could also name and write the three other direction commands without much trouble. If later we want to know how to connect two rooms or move in a particular direction, this test will show us how to do it in no uncertain terms. This test acts as a compileable and executable document that describes the program.

Test Isolation

The act of writing tests before production code often exposes areas in the software that ought to be decoupled. For example, Figure 4-1 shows a simple UML diagram[1] of a payroll application. The `Payroll` class uses the `EmployeeDatabase` class to fetch an `Employee` object. It asks the `Employee` to calculate its pay. Then it passes that pay to the `CheckWriter` object to produce a check. Finally, it posts the payment to the `Employee` object and writes the object back to the database.

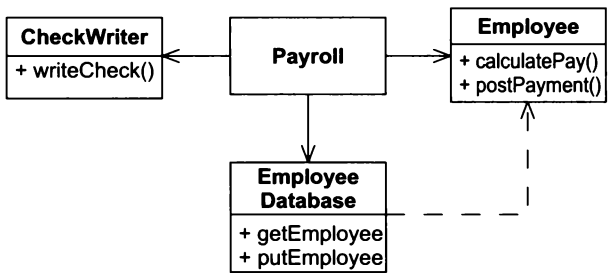


Figure 4-1 Coupled Payroll Model

Presume that we haven't written any of this code yet. So far, this diagram is just sitting on a whiteboard after a quick design session.[2] Now, we need to write the tests that specify the behavior of the `Payroll` object. There are a number of problems associated with writing these tests. First, what database do we use? `Payroll` needs to read from some kind of database. Must we write a fully functioning database before we can test the `Payroll` class? What data do we load into it? Second, how do we verify that the appropriate check got printed? We can't write an automated test that looks on the printer for a check and verifies the amount on it!

The solution to these problems is to use the MOCK OBJECT pattern.[3] We can insert interfaces between all the collaborators of `Payroll` and create test stubs that implement these interfaces.

Figure 4-2 shows the structure. The `Payroll` class now uses interfaces to communicate with the `EmployeeDatabase`, `CheckWriter`, and `Employee`. Three MOCK OBJECTS have been created that implement these interfaces. These MOCK OBJECTS are queried by the `PayrollTest` object to see if the `Payroll` object manages them correctly.

Listing 4-2 shows the intent of the test. It creates the appropriate mock objects, passes them to the `Payroll` object, tells the `Payroll` object to pay all the employees, and then asks the mock objects to verify that all the checks were written correctly and that all the payments were posted correctly.

1. If you don't know UML, there are two appendices that describes it in great detail. See Appendices A and B, starting on page 467.
2. [Jeffries2001].
3. [Mackinnon2000].

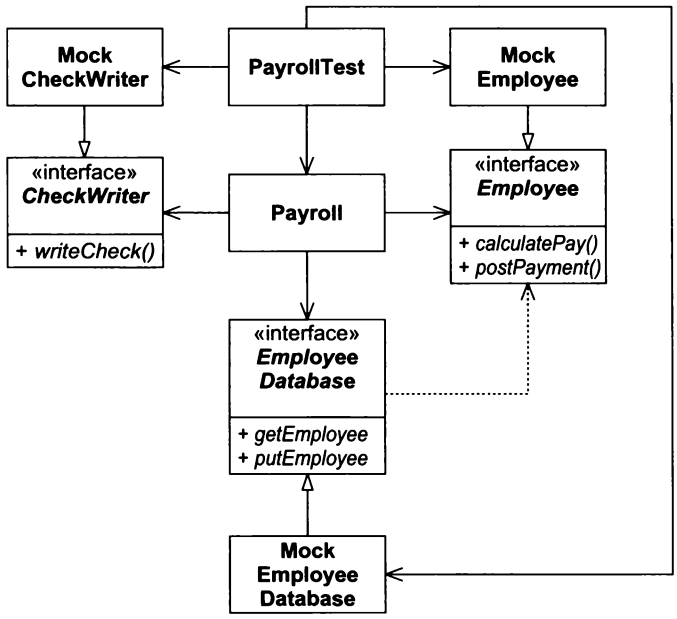


Figure 4-2 Decoupled Payroll using Mock Objects for testing

Listing 4-2

`TestPayroll`

```
public void testPayroll()
{
  MockEmployeeDatabase db = new MockEmployeeDatabase();
  MockCheckWriter w = new MockCheckWriter();
  Payroll p = new Payroll(db, w);
  p.payEmployees();
  assert(w.checksWereWrittenCorrectly());
  assert(db.paymentsWerePostedCorrectly());
}
```

Of course all this test is checking is that `Payroll` called all the right functions with all the right data. It's not actually checking that checks were written. It's not actually checking that a true database was properly updated. Rather, it is checking that the `Payroll` class is behaving as it should in isolation.

You might wonder what the `MockEmployee` is for. It seems feasible that the real `Employee` class could be used instead of a mock. If that were so, then I would have no compunction about using it. In this case, I presumed that the `Employee` class was more complex than needed to check the function of `Payroll`.

Serendipitous Decoupling

The decoupling of `Payroll` is a good thing. It allows us to swap in different databases and check writers, both for the purpose of testing *and* for extension of the application. I think it is interesting that this decoupling was driven by the need to test. Apparently, the need to isolate the module under test forces us to decouple in ways that are beneficial to the overall structure of the program. *Writing tests before code improves our designs.*

A large part of this book is about design principles for managing dependencies. Those principles give you some guidelines and techniques for decoupling classes and packages. You will find these principles most beneficial if you practice them as part of your unit testing strategy. It is the unit tests that will provide much of the impetus and direction for decoupling.

Acceptance Tests



Unit tests are necessary but insufficient as verification tools. Unit tests verify that the small elements of the system work as they are expected to, but they do not verify that the system works properly as a whole. Unit tests are white-box tests[4] that verify the individual mechanisms of the system. Acceptance tests are black-box tests[5] that verify that the customer requirements are being met.

Acceptance tests are written by folks who do not know the internal mechanisms of the system. They may be written directly by the customer or by some technical people attached to the customer, possibly QA. Acceptance tests are programs and are therefore executable. However, they are usually written in a special scripting language created for customers of the application.

Acceptance tests are the ultimate documentation of a feature. Once the customer has written the acceptance tests, which verify that a feature is correct, the programmers can read those acceptance tests to truly understand the feature. So, just as unit tests serve as compileable and executable documentation for the internals of the system, acceptance tests serve as compileable and executable documentation of the features of the system.

Furthermore, the act of writing acceptance tests first has a profound effect upon the architecture of the system. In order to make the system testable, it has to be decoupled at the high architecture level. For example, the user interface (UI) has to be decoupled from the business rules in such a way that the acceptance tests can gain access to those business rules without going through the UI.

In the early iterations of a project, the temptation is to do acceptance tests manually. This is inadvisable because it deprives those early iterations of the decoupling pressure exerted by the need to automate the acceptance tests. When you start the very first iteration, knowing full well that you must automate the acceptance tests, you make very different architectural trade-offs. And, just as unit tests drive you to make superior design decisions in the small, acceptance tests drive you to make superior architecture decisions in the large.

Creating an acceptance testing framework may seem a daunting task. However, if you take only one iteration's worth of features and create only that part of the framework necessary for those few acceptance tests, you'll find it's not that hard to write. You'll also find that the effort is worth the cost.

Example of Acceptance Testing

Consider, again, the payroll application. In our first iteration, we must be able to add and delete employees to and from the database. We must also be able to create paychecks for the employees currently in the database. Fortunately, we only have to deal with salaried employees. The other kinds of employees have been held back until a later iteration.

We haven't written any code yet, and we haven't invested in any design yet. This is the best time to start thinking about acceptance tests. Once again, intentional programming is a useful tool. We should write the acceptance tests the way we think they should appear, and then we can structure the scripting language and payroll system around that structure.

I want to make the acceptance tests convenient to write and easy to change. I want them to be placed in a configuration-management tool and saved so that I can run them anytime I please. Therefore, it makes sense that the acceptance tests should be written in simple text files.

4. A test that knows and depends on the internal structure of the module being tested.
5. A test that does not know or depend on the internal structure of the module being tested.

The following is an example of an acceptance-test script:

```
AddEmp 1429 "Robert Martin" 3215.88
Payday
Verify Paycheck EmpId 1429 GrossPay 3215.88
```

In this example, we add employee number 1429 to the database. His name is "Robert Martin," and his monthly pay is $3215.88. Next, we tell the system that it is payday and that it needs to pay all the employees. Finally, we verify that a paycheck was generated for employee 1429 with a `GrossPay` field of $3215.88.

Clearly, this kind of script will be very easy for customers to write. Also, it will be easy to add new functionality to this kind of script. However, think about what it implies about the structure of the system.

The first two lines of the script are functions of the payroll application. We might call these lines payroll transactions. These are functions that payroll users expect. However, the `Verify` line is not a transaction that the users of payroll would expect. This line is a directive that is specific to the acceptance test.

Thus, our acceptance testing framework will have to parse this text file, separating the payroll transactions from the acceptance-testing directives. It must send the payroll transactions to the payroll application and then use the acceptance-testing directives to query the payroll application in order to verify data.

This already puts architectural stress on the payroll program. The payroll program is going to have to accept input directly from users and also from the acceptance-testing framework. We want to bring those two paths of input together as early as possible. So, it looks as if the payroll program will need a transaction processor that can deal with transactions of the form `AddEmp` and `Payday` coming from more than one source. We need to find some common form for those transactions so that the amount of specialized code is kept to a minimum.

One solution would be to feed the transactions into the payroll application in XML. The acceptance-testing framework could certainly generate XML, and it seems likely that the UI of the payroll system could also generate XML. Thus, we might see transactions that looked like the following:

```
<AddEmp PayType=Salaried>
  <EmpId>1429</EmpId>
  <Name>Robert Martin</Name>
  <Salary>3215.88</Salary>
</AddEmp>
```

These transactions might enter the payroll application through a subroutine call, a socket, or even a batch input file. Indeed, it would be a trivial matter to change from one to the other during the course of development. So during the early iterations, we could decide to read transactions from a file, migrating to an API or socket much later.

How does the acceptance-test framework invoke the `Verify` directive? Clearly it must have some way to access the data produced by the payroll application. Once again, we don't want the acceptance-testing framework to have to try to read the writing on a printed check, but we can do the next best thing.

We can have the payroll application produce its paychecks in XML. The acceptance-testing framework can then catch this XML and query it for the appropriate data. The final step of printing the check from the XML may be trivial enough to handle through manual acceptance tests.

Therefore, the payroll application can create an XML document that contains all the paychecks. It might look like this:

```
<Paycheck>
  <EmpId>1429</EmpId>
  <Name>Robert Martin</Name>
  <GrossPay>3215.88</GrossPay>
</Paycheck>
```

Clearly, the acceptance-testing framework can execute the `Verify` directive when supplied with this XML.

Once again, we can spit the XML out through a socket, through an API, or into a file. For the initial iterations, a file is probably easiest. Therefore, the payroll application will begin its life reading XML transactions in from a file and outputting XML paychecks to a file. The acceptance-testing framework will read transactions in text form, translating them to XML and writing them to a file. It will then invoke the payroll program. Finally, it will read the output XML from the payroll program and invoke the `Verify` directives.

Serendipitous Architecture

Notice the pressure that the acceptance tests placed upon the architecture of the payroll system. The very fact that we considered the tests first led us to the notion of XML input and output very quickly. This architecture has decoupled the transaction sources from the payroll application. It has also decoupled the paycheck printing mechanism from the payroll application. These are good architectural decisions.

Conclusion

The simpler it is to run a suite of tests, the more often those tests will be run. The more the tests are run, the faster any deviation from those tests will be found. If we can run all the tests several times a day, then the system will never be broken for more than a few minutes. This is a reasonable goal. We simply don't allow the system to backslide. Once it works to a certain level, it never backslides to a lower level.

Yet verification is just one of the benefits of writing tests. Both unit tests and acceptance tests are a form of documentation. That documentation is compileable and executable; therefore, it is accurate and reliable. Moreover, these tests are written in unambiguous languages that are made to be readable by their audience. Programmers can read unit tests because they are written in their programming language. Customers can read acceptance tests because they are written in a language that they themselves designed.

Possibly the most important benefit of all this testing is the impact it has on architecture and design. To make a module or an application testable, it must also be decoupled. The more testable it is, the more decoupled it is. The act of considering comprehensive acceptance and unit tests has a profoundly positive effect upon the structure of the software.

Bibliography

1. Mackinnon, Tim, Steve Freeman, and Philip Craig. Endo-Testing: Unit Testing with Mock Objects. *Extreme Programming Examined.* Addison–Wesley, 2001.
2. Jeffries, Ron, et al., *Extreme Programming Installed.* Upper Saddle River, NJ: Addison–Wesley, 2001.

5

Refactoring



The only factor becoming scarce in a world of abundance is human attention.

—Kevin Kelly, in *Wired*

This chapter is about human attention. It is about paying attention to what you are doing and making sure you are doing your best. It is about the difference between getting something to work and getting something right. It is about the value we place in the structure of our code.

In his classic book, *Refactoring*, Martin Fowler defines refactoring as "...the process of changing a software system in such a way that it does not alter the external behavior of the code yet improves its internal structure."[1] But why would we want to improve the structure of working code? What about the old saw, "if it's not broken, don't fix it!"?

Every software module has three functions. First, there is the function it performs while executing. This function is the reason for the module's existence. The second function of a module is to afford change. Almost all modules will change in the course of their lives, and it is the responsibility of the developers to make sure that such changes are as simple as possible to make. A module that is hard to change is broken and needs fixing, even though it works. The third function of a module is to communicate to its readers. Developers unfamiliar with the module should be able to read and understand it without undue mental gymnastics. A module that does not communicate is broken and needs to be fixed.

1. [Fowler99], p. xvi.

What does it take to make a module easy to read and easy to change? Much of this book is dedicated to principles and patterns whose primary goal is to help you create modules that are flexible and adaptable. However, it takes something more than just principles and patterns to make a module that is easy to read and change. It takes attention. It takes discipline. It takes a passion for creating beauty.

Generating Primes: A Simple Example of Refactoring[2]

Consider the code in Listing 5–1. This program generates prime numbers. It is one big function with many single letter variables and comments to help us read it.

Listing 5-1

GeneratePrimes.java version 1

```
/**
 * This class generates prime numbers up to a user-specified
 * maximum.  The algorithm used is the Sieve of Eratosthenes.
 * <p>
 * Eratosthenes of Cyrene, b. c. 276 BC, Cyrene, Libya --
 * d. c. 194, Alexandria.  The first man to calculate the
 * circumference of the Earth.  Also known for working on
 * calendars with leap years, he ran the library at Alexandria.
 * <p>
 * The algorithm is quite simple.  Given an array of integers
 * starting at 2.  Cross out all multiples of 2.  Find the next
 * uncrossed integer, and cross out all of its multiples.
 * Repeat until you have passed the square root of the maximum
 * value.
 *
 * @author Robert C. Martin
 * @version 9 Dec 1999 rcm
 */
import java.util.*;

public class GeneratePrimes
{
  /**
   * @param maxValue is the generation limit.
   */
  public static int[] generatePrimes(int maxValue)
  {
    if (maxValue >= 2) // the only valid case
    {
      // declarations
      int s = maxValue + 1; // size of array
      boolean[] f = new boolean[s];
      int i;

      // initialize array to true.
      for (i = 0; i < s; i++)
        f[i] = true;
```

2. I initially wrote this program for XP Immersion I using tests written by Jim Newkirk. Kent Beck and Jim Newkirk refactored it in front of the students. I have tried to recreate that refactoring here.

```
      // get rid of known non-primes
      f[0] = f[1] = false;

      // sieve
      int j;
      for (i = 2; i < Math.sqrt(s) + 1; i++)
      {
        if (f[i]) // if i is uncrossed, cross its multiples.
        {
          for (j = 2 * i; j < s; j += i)
            f[j] = false; // multiple is not prime
        }
      }

      // how many primes are there?
      int count = 0;
      for (i = 0; i < s; i++)
      {
        if (f[i])
          count++; // bump count.
      }

      int[] primes = new int[count];

      // move the primes into the result
      for (i = 0, j = 0; i < s; i++)
      {
        if (f[i])             // if prime
          primes[j++] = i;
      }

      return primes;  // return the primes
    }
    else // maxValue < 2
      return new int[0]; // return null array if bad input.
  }
}
```

The unit test for `GeneratePrimes` is shown in Listing 5–2. It takes a statistical approach, checking to see if the generator can generate primes up to 0, 2, 3, and 100. In the first case there should be no primes. In the second there should be one prime, and it should be 2. In the third there should be two primes, and they should be 2 and 3. In the last case there should be 25 primes, the last of which is 97. If all these tests pass, then I make the assumption that the generator is working. I doubt this is foolproof, but I can't think of a reasonable scenario where these tests would pass and yet the function would fail.

Listing 5-2

TestGeneratePrimes.java

```
import junit.framework.*;
import java.util.*;

public class TestGeneratePrimes extends TestCase
{
  public static void main(String args[])
```

```
  {
      junit.swingui.TestRunner.main(
        new String[] {"TestGeneratePrimes"});
  }
  public TestGeneratePrimes(String name)
  {
    super(name);
  }

  public void testPrimes()
  {
    int[] nullArray = GeneratePrimes.generatePrimes(0);
    assertEquals(nullArray.length, 0);

    int[] minArray = GeneratePrimes.generatePrimes(2);
    assertEquals(minArray.length, 1);
    assertEquals(minArray[0], 2);

    int[] threeArray = GeneratePrimes.generatePrimes(3);
    assertEquals(threeArray.length, 2);
    assertEquals(threeArray[0], 2);
    assertEquals(threeArray[1], 3);

    int[] centArray = GeneratePrimes.generatePrimes(100);
    assertEquals(centArray.length, 25);
    assertEquals(centArray[24], 97);
  }
}
```

To help me refactor this program, I am using the *Idea* refactoring browser from *IntelliJ*. This tool makes it trivial to extract methods and rename variables and classes.

It seems pretty clear that the main function wants to be three separate functions. The first initializes all the variables and sets up the sieve. The second actually executes the sieve, and the third loads the sieved results into an integer array. To expose this structure more clearly in Listing 5–3, I extracted those functions into three separate methods. I also removed a few unnecessary comments and changed the name of the class to `PrimeGenerator`. The tests all still ran.

Extracting the three functions forced me to promote some of the variables of the function to static fields of the class. I think this clarifies which variables are local and which have wider influence.

Listing 5-3

PrimeGenerator.java, version 2

```
/**
 * This class generates prime numbers up to a user-specified
 * maximum.  The algorithm used is the Sieve of Eratosthenes.
 * Given an array of integers starting at 2:
 * Find the first uncrossed integer, and cross out all its
 * multiples.  Repeat until the first uncrossed integer exceeds
 * the square root of the maximum value.
 */
import java.util.*;
```

```
public class PrimeGenerator
{
  private static int s;
  private static boolean[] f;
  private static int[] primes;

  public static int[] generatePrimes(int maxValue)
  {
    if (maxValue < 2)
      return new int[0];
    else
    {
      initializeSieve(maxValue);
      sieve();
      loadPrimes();
      return primes;  // return the primes
    }
  }

  private static void loadPrimes()
  {
    int i;
    int j;

    // how many primes are there?
    int count = 0;
    for (i = 0; i < s; i++)
    {
      if (f[i])
        count++; // bump count.
    }

    primes = new int[count];

    // move the primes into the result
    for (i = 0, j = 0; i < s; i++)
    {
      if (f[i])             // if prime
        primes[j++] = i;
    }
  }

  private static void sieve()
  {
    int i;
    int j;
    for (i = 2; i < Math.sqrt(s) + 1; i++)
    {
      if (f[i])  // if i is uncrossed, cross out its multiples.
      {
        for (j = 2 * i; j < s; j += i)
          f[j] = false; // multiple is not prime
      }
    }
  }
```